Functional Requirements Document (FRD)

Document Code: FRD-DB-2026-V1.0

Project: Duplicate Bin Ecosystem

Author: Senior Technical Business Analyst

Scope: Detailed Functional Specifications, Interaction Flows & Acceptance Criteria

Standard: IEEE 830 Specification with Gherkin Acceptance Criteria

1. Feature Index & Module Map

Feature ID	Module	Feature Name	Primary Actor	Target Release
FEAT-01	Core Engine	Two-Pass Size Pre-Filter & Hash Detection	End User / IT Admin	v0.2.0
FEAT-02	Core Engine	Union-Find Transitive Group Clustering	End User	v0.2.0
FEAT-03	Selection	Automated Smart Selection Strategies (8 Modes)	End User	v0.2.0
FEAT-04	Archive	Safe Archive Quarantine & Instant Restore (`Ctrl+Z`)	End User	v0.2.0
FEAT-05	Cache	Persistent SQLite WAL Hash Caching	System Engine	v0.2.0
FEAT-06	UI Layer	Drag & Drop Folder Ingestion & Split-Pane Previews	End User	v0.2.0
FEAT-07	SaaS Backend	6-Digit Device Pairing & Persistent Token Exchange	IT Admin / Manager	v1.0.0 (SaaS)
FEAT-08	SaaS Backend	Real-Time WebSocket Scan Dispatch & Progress Relay	Team Manager	v1.0.0 (SaaS)
FEAT-09	Billing	Deduplication Credit Ingestion &	Account Owner	v1.0.0 (SaaS)

		Deduplication Credit Ingestion &		

Feature ID	Module	Feature Name	Primary Actor	Target Release
		Plan Upgrades		

2. Detailed Functional Specifications

2.1 FEAT-01: Two-Pass Size Pre-Filter & Cryptographic Hashing

- **Requirement ID:** FR-001, FR-002
- **Module:** Core Detection Engine (`duplicate_bin/core/detection.py`, `hashing.py`)
- **Actor:** Prosumer / IT Administrator
- **Preconditions:** Target directory path exists and user has OS read permissions.
- **Trigger:** User clicks "Start Scan" or executes `run.py scan <path>`.
- **Inputs:** Target directory path, algorithm (`sha256`, `md5`, `blake2b`, `crc32`), chunk size (4KB - 1MB adaptive).
- **Outputs:** Candidate file list grouped by size, cryptographic digests calculated for non-unique size buckets.
- **Business Rules:** Files with unique byte sizes across the scanned corpus are excluded immediately in Pass 1 ($O(n)$ optimization). Hashing is executed strictly on size collisions (Pass 2).
- **Main Flow:**
  1. Engine invokes `collect_files(path)`.
  2. Engine records each file's size using `os.stat().st_size`.
  3. Engine groups files into a hash table keyed by file size.
  4. Buckets where `len(files) == 1` are discarded.
  5. For remaining multi-file buckets, the engine streams byte chunks through the selected hash algorithm.
  6. Generated hashes are returned with file metadata.
- **Alternative Flow (Cache Hit):**
- If a file's `(path, size, mtime)` matches an entry in the SQLite cache, the cached hash is returned immediately, bypassing disk read.
- **Exception Flow (Permission Denied):**

- If a file or folder raises `PermissionError`, the exception is logged, the file is skipped, and scanning continues uninterrupted.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Rapid pre-filtering of unique files Given a target folder containing 5,000 files where 4,800 have unique byte sizes When the user initiates a duplicate detection scan Then the engine must only compute cryptographic hashes for the 200 files sharing size collisions And the 4,800 unique files must be excluded within Pass 1 processing in O(n) time.
```

2.2 FEAT-02: Union-Find Transitive Group Clustering

- **Requirement ID:** FR-003
- **Module:** Core Detection Engine (`duplicate_bin/core/detection.py`)
- **Actor:** Core Processing Engine
- **Preconditions:** Hash calculation completed for candidate buckets.
- **Trigger:** Completion of Pass 2 hashing.
- **Inputs:** List of `(file_path, hash_value)` tuples.
- **Outputs:** Duplicate clusters where each group contains ≥ 2 identical files.
- **Business Rules:** If File A equals File B, and File B equals File C, all three must be unified into group ${A, B, C}$ without group fragmentation or collision bugs.
- **Main Flow:**
  1. Engine initializes a disjoint-set (Union-Find) structure with path compression.
  2. For files with identical hashes, the engine performs `union(file_A, file_B)`.
  3. Engine collapses sets using `find()` to determine canonical root representatives.
  4. Engine outputs grouped clusters with calculated group size and recoverable byte count.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Transitive clustering of duplicate files across subdirectories Given three identical 50 MB video files: "vid.mov", "copy_vid.mov", and "backup/vid.mov" When the detection engine processes the file signatures Then all 3 files must be grouped into a single Duplicate Group cluster And the group's recoverable storage must equal exactly 100 MB (preserving 1 original of 50 MB).
```

2.3 FEAT-03: Automated Smart Selection Strategies (8 Modes)

- **Requirement ID:** FR-004
- **Module:** Selection Engine (`duplicate_bin/core/selection.py`)
- **Actor:** Prosumer / IT Administrator
- **Preconditions:** Duplicate clusters identified and presented in memory.
- **Trigger:** User selects a strategy from the dropdown or configures CLI parameter `--strategy`.
- **Inputs:** `SelectionStrategy` enum (KEEP_NEWEST, KEEP_OLDEST, KEEP_LARGEST, KEEP_SMALLEST, KEEP_BEST_QUALITY, KEEP_SHORTEST_PATH, KEEP_DEEPEST_PATH, CUSTOM_PRIORITY).
- **Outputs:** Boolean flag `is_selected_for_removal` set on each duplicate instance.
- **Business Rules:**
- **Master Preservation Rule:** Exactly one master file must remain unselected per group.
- In `KEEP_BEST_QUALITY`, RAW image extensions (`.cr2`, `.nef`, `.arw`) and higher-resolution/bitrate formats take precedence over compressed formats (`.jpg`, `.mp3`).
- **Main Flow:**
  1. Selection engine iterates through each duplicate group.
  2. Selection engine evaluates file attributes according to the chosen strategy.
  3. The highest-scoring file is designated as the master (`keep=True`).
  4. All remaining redundant files in the group are marked for removal (`keep=False`).
  5. UI updates checkbox state across the tree view.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Automated selection using Keep Best Quality Given a duplicate group containing "photo.cr2" (RAW, 28 MB) and "photo.jpg" (Compressed, 4 MB) When the user applies the "KEEP_BEST_QUALITY" strategy Then "photo.cr2" must be marked as the master file to retain And "photo.jpg" must be marked for removal And the user must not be required to manually check individual boxes.
```

2.4 FEAT-04: Safe Archive Quarantine & Instant Restore

- **Requirement ID:** FR-006, FR-007
- **Module:** Archive Engine (`duplicate_bin/engine/archive.py`)
- **Actor:** Prosumer / Enterprise User

- **Preconditions:** At least one duplicate file marked for removal.
- **Trigger:** User clicks "Archive Selected" or presses `Delete`.
- **Inputs:** List of absolute file paths to quarantine.
- **Outputs:** Files relocated to `~/.duplicate_bin/archive/<timestamp>/`, audit log entry written, instant undo buffer populated.
- **Business Rules:** Destructive permanent OS deletion (`os.remove`) is prohibited by default. Files must be safely moved preserving relative subdirectory paths.
- **Main Flow:**
  1. User confirms archive action in dialog.
  2. Engine creates a unique timestamped quarantine directory inside `~/.duplicate_bin/archive/`.
  3. For each selected file, engine resolves canonical source path and safely moves the file using `shutil.move()`.
  4. Engine persists a manifest mapping original source paths to quarantine paths.
  5. User interface enables the "Restore" button (`Ctrl+Z`).
- **Alternative Flow (Undo / Restore):**
- User triggers `Ctrl+Z`. Engine reads the latest manifest, moves files back to their exact original locations, and clears the quarantine directory.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Zero-risk archiving and rollback Given 5 duplicate files successfully archived into the quarantine directory When the user presses Ctrl+Z or clicks "Restore Files" Then all 5 files must be returned to their exact original directory locations And file sizes and modification timestamps must remain intact And the UI must confirm 0 bytes lost.
```

2.5 FEAT-07: 6-Digit Device Pairing & Persistent Token Exchange

- **Requirement ID:** FR-010
- **Module:** SaaS Backend & Agent Client (`backend/app/main.py`, `agent_client.py`)
- **Actor:** Team Manager & Local Agent Daemon
- **Preconditions:** User is logged into SaaS web portal; agent daemon is installed on workstation.
- **Trigger:** User requests pairing code on web portal (`GET /api/devices/pairing-code`).
- **Inputs:** Generated 6-digit random code, client machine hostname, operating system name.

- **Outputs:** Persistent 64-character hex `pairing_token` stored on agent; device registered in database.
- **Business Rules:** Pairing code is valid for exactly 300 seconds (5 minutes). Once paired or expired, code is purged from memory.
- **Main Flow:**
  1. Portal calls `/api/devices/pairing-code` and renders "839201" with a 5-minute countdown.
  2. User runs `run.py pair 839201` on workstation.
  3. Agent issues `POST /api/devices/pair` transmitting pairing code, unique UUID `device_id`, hostname, and OS.
  4. Backend verifies code is present in `PAIRING_CODES` and within 300s window.
  5. Backend generates `pairing_token = secrets.token_hex(32)`, creates or updates `Device` record in DB, and removes pairing code.
  6. Agent receives token and persists it in `~/.duplicate_bin/agent_config.json`.
- **Exception Flow (Expired Code):**
- If elapsed time exceeds 300s, backend returns `400 Bad Request: Pairing code expired`.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Secure workstation pairing via 6-digit code Given a valid pairing code "492018" generated 60 seconds ago When the administrator executes "python run.py pair 492018" on an office workstation Then the backend must register the device under the administrator's account And the backend must return a unique 64-character pairing token And subsequent requests using the same pairing code must be rejected.
```

2.6 FEAT-09: Deduplication Credit Ingestion & Plan Upgrades

- **Requirement ID:** FR-012, BR-003
- **Module:** SaaS Billing Engine (`backend/app/main.py`, `models.py`)
- **Actor:** SaaS Account Owner
- **Preconditions:** Registered user account with valid JWT token.
- **Trigger:** Web portal purchase request or scan telemetry completion.
- **Inputs:** `bytes_to_buy` or `plan_name`.
- **Outputs:** Updated user record with new `credit_bytes` balance or subscription plan.
- **Business Rules:** New accounts receive a default free tier credit of 10 GB ($10,737,418,240$ bytes). Telemetry from completed scans decrements credit balance by `bytes_recoverable`.

- **Main Flow (Credit Purchase):**
  1. User calls `POST /api/billing/buy-credits` with `{"bytes_to_buy": 107374182400}` (100 GB).
  2. Backend increments `current_user.credit_bytes` by the requested byte amount.
  3. Backend commits transaction and returns updated credit balance.
- **Main Flow (Credit Deduction):**
  1. Agent completes scan and posts telemetry to `POST /api/scans/telemetry`.
  2. If scan status is `completed`, backend subtracts `bytes_recoverable` from `user.credit_bytes`.
- **Acceptance Criteria (Gherkin):**

  `gherkin Scenario: Accurate credit ledger deduction on scan completion Given a user account with 10 GB of available credit bytes When a paired device completes a scan recovering 3 GB of duplicate files Then the user's credit ledger must be debited by exactly 3 GB And the remaining balance must display as 7 GB on the central dashboard.`